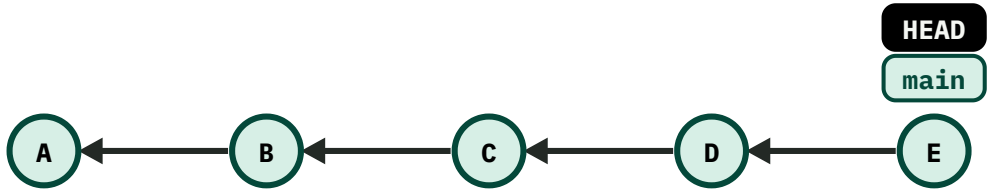
UNIFAAT 2026.2

12 - Branches, Fork e Pull Request

Prof. Luan Lourenço

Revisão rápida: o commit

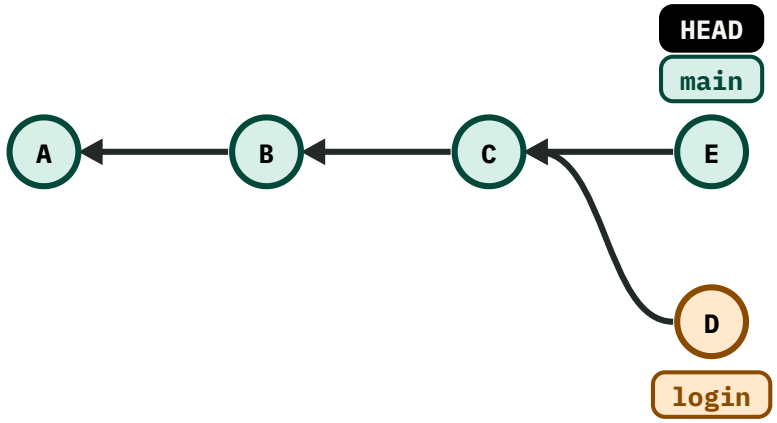


`git add . && git commit -m "E"`

Mais um. Uma linha reta, um commit depois do outro: **uma única linha do tempo**. Hoje vamos ter mais de uma.

Commit = **foto do projeto + ponteiro para o pai**. O fluxo é sempre `editar` → `git add` → `git commit`.

Branch: o multiverso da linha do tempo



`git commit -m "E"` Agora são **dois universos paralelos**: a main com o E, o login com o D. Os dois existem ao mesmo tempo, e nenhum estragou o outro.

Uma branch **não é cópia de arquivos**: é um ponteiro (um texto com o hash de um commit). Criar é de graça.

`checkout` e `switch`: viajar entre os universos

```
# o antigo: git checkout
```
`git checkout login`
`git checkout -b login` (cria e entra)

```
# canivete suíço
```
troca de branch, restaura arquivo, vai para commit solto

```
# o moderno: git switch (Git 2.23)
```
`git switch login`
`git switch -c login` (cria e entra)

```
# faz uma coisa só
```
troca de branch. Arquivo: `git restore`

projeto/

```
# cria a branch e entra nela (o HEAD muda de lugar)
$ git switch -c login
# o HEAD aponta para a branch nova
$ cat .git/HEAD
ref: refs/heads/login
```

Os dois funcionam e aparecem por aí. Use **switch** (diz o que faz) e saiba ler **checkout**. Ao trocar, os arquivos da pasta mudam: **commite antes**.

`git push`: escolhemos o remote e a branch

```
# o formato
git push <remote> <branch>

# o mais comum
remote origin, branch main
```

```
# mas é escolha, e dá para mudar
outro remote: upstream, origin...
outra branch: login, meu-tf...
```

projeto/

```
# envia a branch login para o remote origin (-u guarda o destino)
$ git push -u origin login
# lista também as branches que existem nos remotes
$ git branch -a
```

- O galho **só existe no GitHub depois do primeiro push** dele
- O `push` envia a **branch em que você está**: confira antes

Chave SSH: na conta inteira ou em um repositório só

```
# chave da conta
Settings → SSH and GPG keys
alcance: tudo que a conta acessa
se vazar: age como você em tudo
```

```
# chave do repositório (deploy key)
repo → Settings → Deploy keys
alcance: só aquele repositório
somente leitura por padrão (escrita só se marcar)
```

- → **Deploy key:** para uma máquina que só precisa baixar o código (servidor, CI)
- → **Chave da conta:** para vocês, no próprio computador, com vários repositórios (é a da Aula 05)

Princípio do **menor privilégio**: cada chave com só o acesso de que precisa.

Sempre SSH, e não HTTPS

projeto/

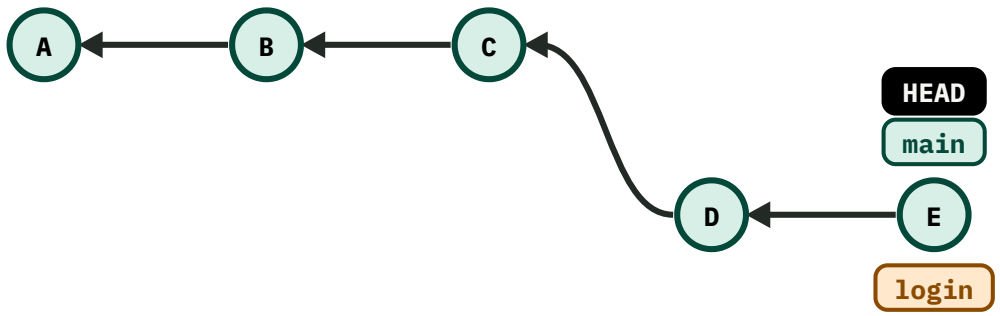
```
# a URL SSH: a autenticação é a chave, sem digitar nada
git@github.com:luan-tavares/unifaat-es-2026.git
# a URL HTTPS: pede credencial
https://github.com/luan-tavares/unifaat-es-2026.git
# clonou por HTTPS? troque o remote, sem clonar de novo
$ git remote set-url origin git@github.com:SEU-USUARIO/unifaat-es-2026.git
$ ssh -T git@github.com
```

- **SSH:** configura uma vez e `push` / `pull` não pedem nada
- **HTTPS:** desde 2021 o GitHub não aceita mais a senha da conta: exige um **token**, um segredo a mais para guardar

---

Na disciplina: **sempre SSH**. A URL do clone é a escolha do protocolo.

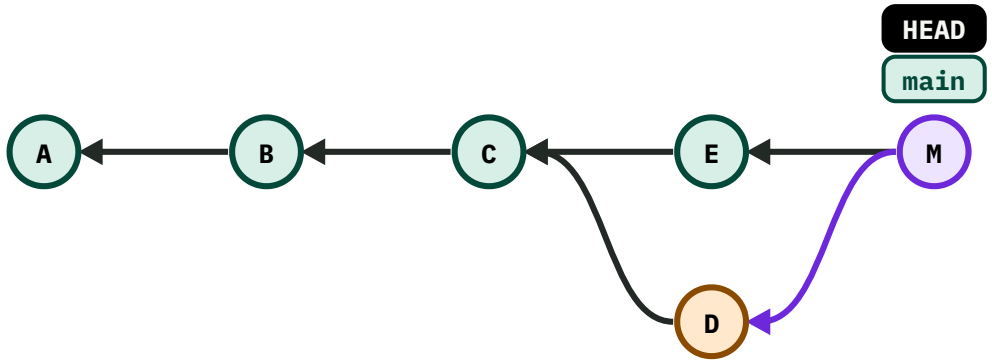
Merge: trazer o galho para onde o HEAD está



Os commits do galho agora fazem parte da **main** (mudaram de cor). O galho **login** continua existindo, apontando para o E.

Se a main **não andou**, o merge é só mover o ponteiro (**fast-forward**): nenhum commit novo.

Merge: quando os dois lados andaram



`git branch -d login` Mesclar **não apaga o galho**. Apagar é outra decisão e só remove o **ponteiro** login: o commit D continua no histórico, alcançável pelo M.

O commit de merge tem **dois pais**. Mesclar **não apaga o galho**: apagar é outra decisão.

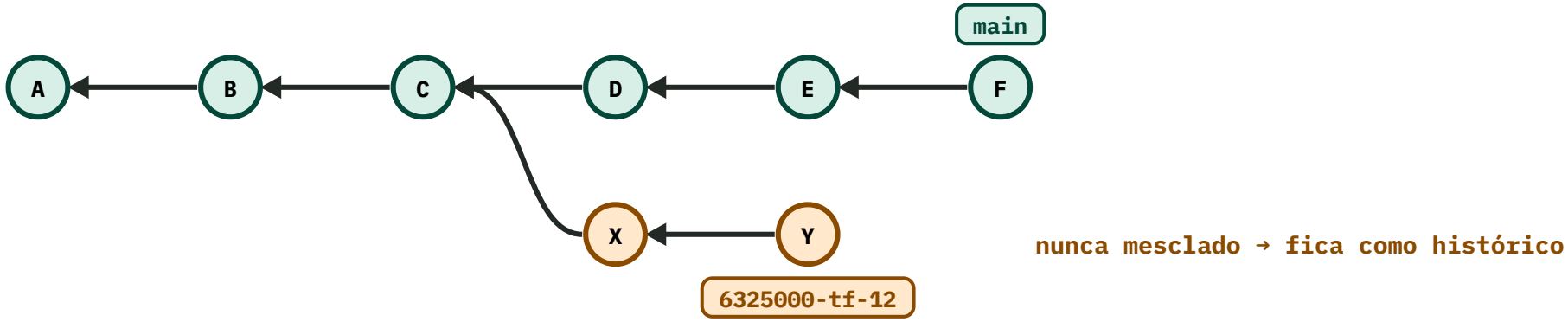
Merge: os cuidados

- **Confira o destino:** o merge traz o galho para a branch **atual** (`git branch`)
- **Commite antes** e **atualize o tronco** (`git pull`) antes de mesclar
- **Mesclar no main é publicar:** em equipe, passa por revisão (o Pull Request)
- **Conflito:** mesma linha mudada nos dois lados. Um humano decide
- **Não reescreva histórico publicado** (`push --force`)

projeto/

```
# conflito: edite o arquivo, apague os marcadores e conclua
<<<<<<< HEAD   (a branch atual)
=======              (separa os dois lados)
>>>>>>> login  (o galho que entra)
# desistiu? volta ao estado de antes
$ git merge --abort
```

Tronco, galhos e o galho que nunca é mesclado



O galho **nunca é mesclado**: não volta para a main. Mas também **não some**: fica no GitHub como **histórico**, o registro da entrega.

Tronco = a `main` (o oficial). **Galho** = branch de trabalho. Na disciplina, o galho do aluno **nunca é mesclado**: fica como **histórico**.

Fork x clone x pull

```
# fork
onde: no GitHub, de conta para conta
faz: copia o repositório do professor para a sua conta
quando: 1 vez
como: botão Fork (não é comando do Git)
```

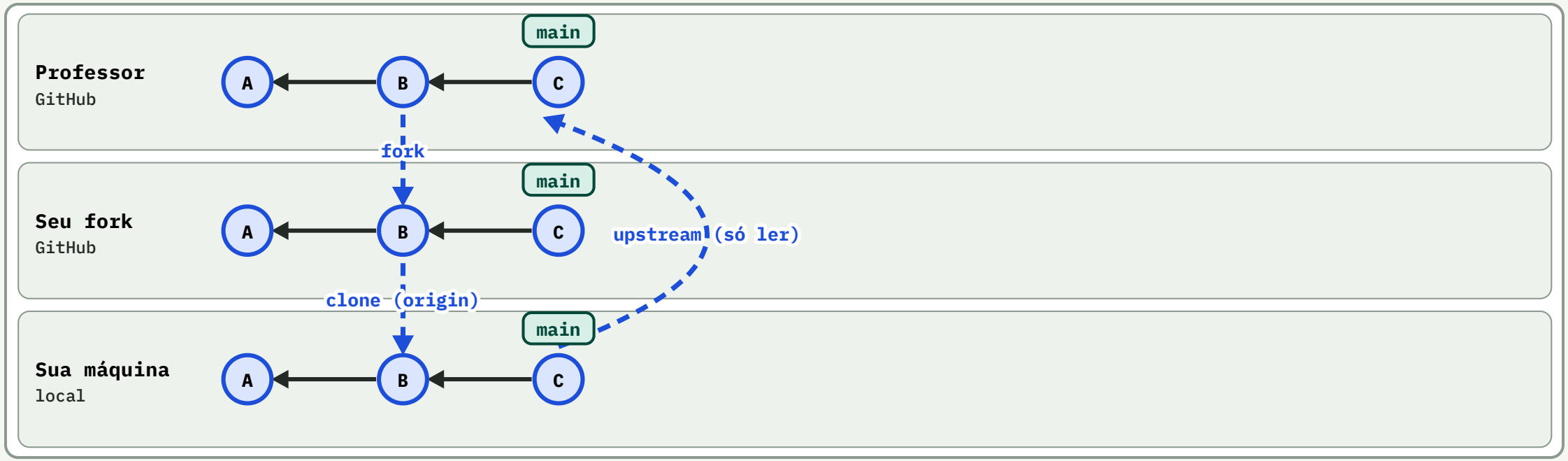
```
# clone
onde: do GitHub para a sua máquina
faz: baixa o repositório inteiro e já configura o origin
quando: 1 vez por máquina
como: git clone <url>
```

```
# pull
onde: de um remote para a sua branch local
faz: traz só o que é novo (fetch + merge)
quando: todo dia, em repo já clonado
como: git pull --ff-only upstream main
```

Um não impede o outro: é um caminho só, **fork → clone → pull**. **Por que o fork vem primeiro?**

- → **Escrita:** sem fork, você clona e dá `pull` do professor, mas o `push` é **recusado**. O fork é o **seu** lugar com permissão
- → **Vínculo com o original:** o fork lembra de onde veio. Por isso existem o **Sync fork**, o aviso "commits behind" e o **PR** para o repositório do professor
- → **Isolamento:** o `main` do professor fica intocado; o seu trabalho mora no seu fork
- → **Cópia sua no GitHub:** o TF fica guardado na sua conta (backup e portfólio)

Fork: a sua cópia do repositório

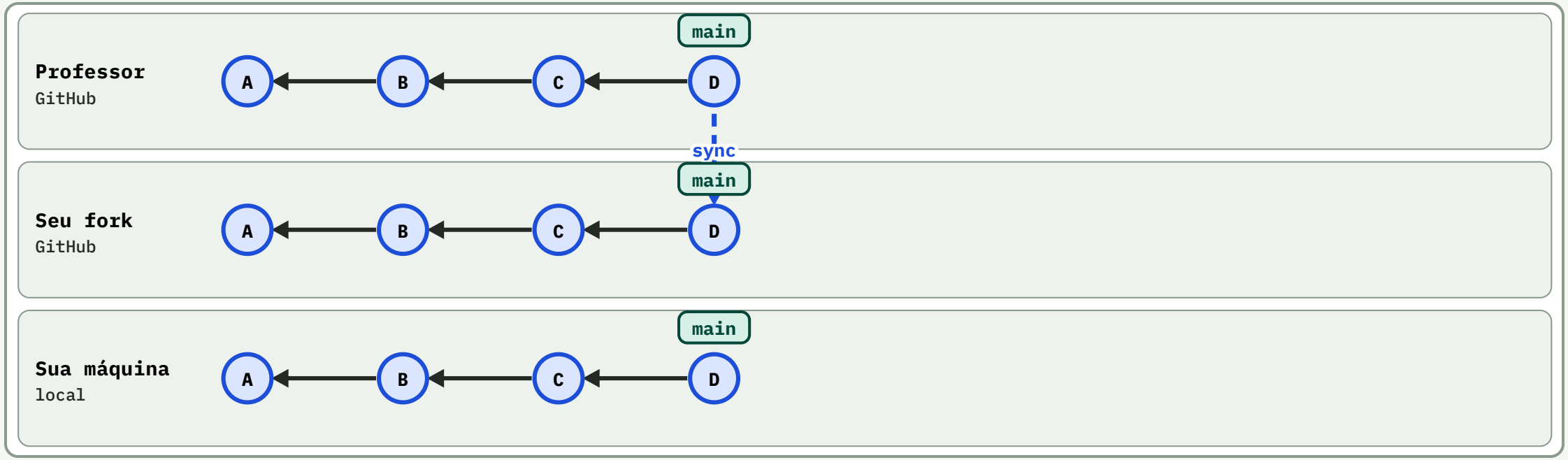



```
git remote add upstream git@github.com:luan-tavares/...
```

O **upstream** é o repositório do professor cadastrado só para **ler**: de lá vêm as aulas novas.

Fork **não é clone**: o fork é de conta para conta (no GitHub); o clone é do GitHub para a sua máquina. No `main` do professor, **só o professor escreve**.

Sync e pull: as aulas novas

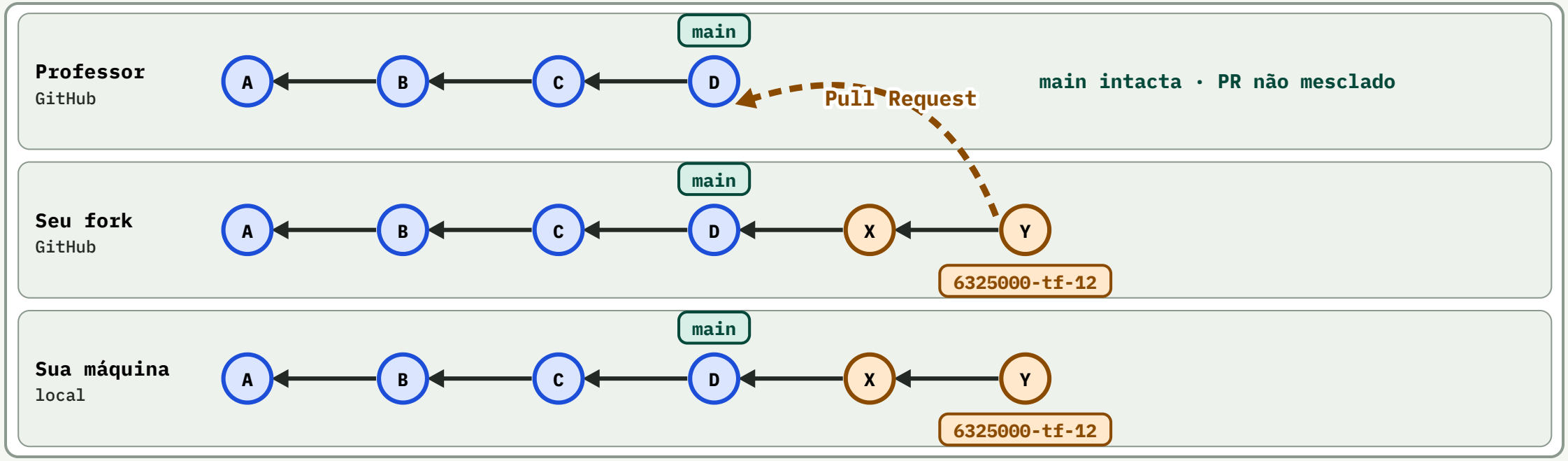


botão Sync fork (no GitHub)

O **sync** do GitHub atualiza o **fork** na nuvem (opcional). Para o TF, o que importa é o pull: o seu **main** é só uma cópia do professor.

O fork **não se atualiza sozinho**. O seu `main` só recebe `pull` do `upstream`. `--ff-only` = **só avanço limpo** (fast-forward): se alguém mexeu no seu `main`, o Git **recusa** em vez de criar um merge.

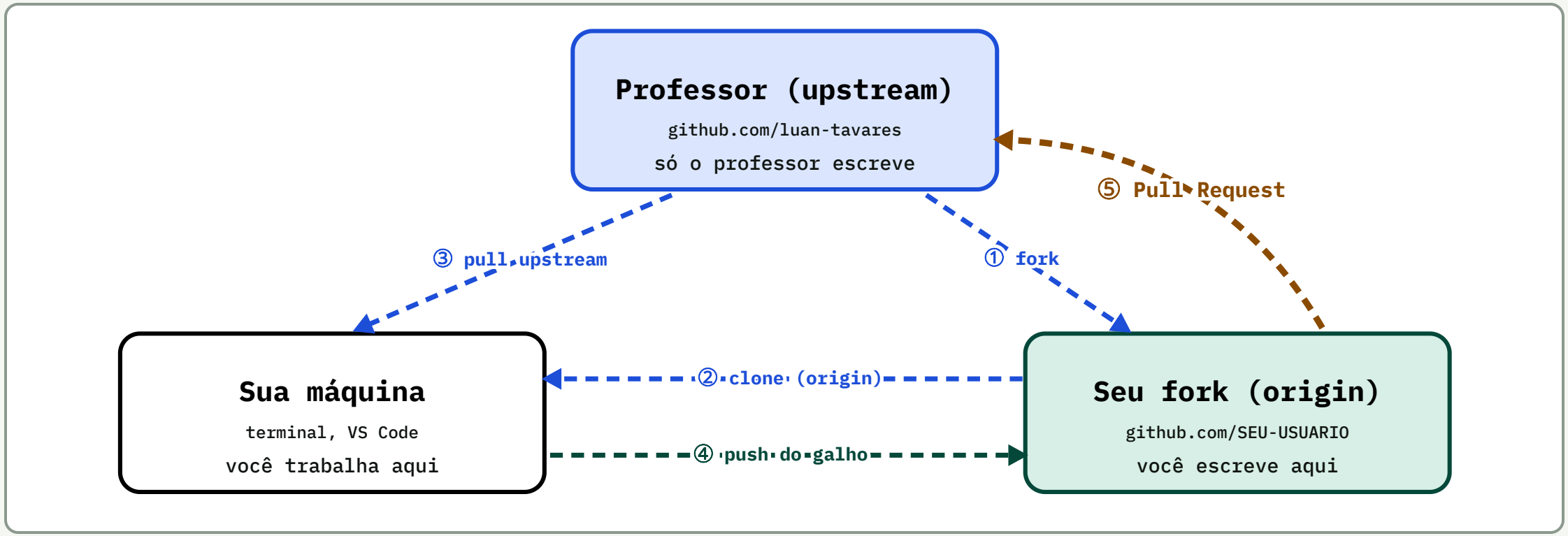
Push em outro galho e Pull Request



O PR **nunca é mesclado**: a main do professor não muda e o galho fica no GitHub como **histórico** da entrega.

O PR é um **pedido**: "professor, puxe o meu galho". **Não é comando do Git**: o `push` é no terminal, o PR é **no site do GitHub** (interface web).

O fluxo inteiro: fork, pull, push e PR



botão Compare & pull request (site do GitHub)	⑤ **Pull Request:** o **pedido** para o professor puxar o seu galho. Ele corrige e fecha, sem mesclar.

Azul: receber (fork, clone, pull). **Verde:** enviar (push, só para o **seu** fork). **Laranja:** pedir (PR para o professor). Você escreve no fork; o professor só **recebe o pedido**.

Os comandos, e o pedido que aparece

unifaat-es-2026/

```
# 1) traz a aula nova do professor para o seu main
# --ff-only = "fast-forward only": só aceita avanço limpo (o main é cópia do professor).
# se você mexeu no main, ele RECUSA, em vez de criar um merge surpresa
$ git switch main && git pull --ff-only upstream main
# 2) cria o galho do TF, com o seu RA, e entra nele
$ git switch -c 6325000-tf-12
# 3) cria a estrutura do TF: RA/tf-12/RA.txt (com nome, RA e GitHub) e grava no galho
$ mkdir -p 6325000/tf-12
$ git add -A && git commit -m "TF-12"
# 4) envia o galho para o SEU fork (origin), nunca para o professor
$ git push -u origin 6325000-tf-12
```

5) depois do push, no site do seu fork, o GitHub mostra:



Clique no botão e confira: **base** = repositório do professor (`main`), **head** = o seu fork e o seu galho. Título `TF-12 · RA · Nome` → **Create pull request**.

tarefa

TF 12: o seu PR é a entrega

- **Branch** `RA-tf-12` (ex.: `6325000-tf-12`), criada a partir do `main` atualizado
- **Estrutura:** `RA/tf-12/RA.txt` (ex.: `6325000/tf-12/6325000.txt`) com **nome, RA e usuário do GitHub**
- **Pull Request** aberto **no site do GitHub**, do seu fork para o `main` do professor. Título `TF-12 · RA · Nome`
- **O TF só é válido se o PR aparecer** na lista de Pull requests do professor. `push` sem PR não é entrega

Descrição completa do TF (README): `https://github.com/luan-tavares/unifaat-es-2026/tree/main/aula-12-branches-fork-pr`

Prazo: **quarta-feira, 18/11**.